HW #3: Dungeons and Dragons

Faculty of Computer and Information Science, Ben-Gurion University of the Negev Spring 2026

Submission in pairs (groups of two students)

Overview

In this assignment you will implement a single-player multi-level version of a *dungeons and dragons* board game.

You are trapped in a dungeon, surrounded by enemies. Your goal is to fight your way through them and get to the next level of the dungeon. Once you complete all the levels, you win the game.

The program will be checked for correctness, complying to OOP principles and coding conventions.



Game Description

The game is played on a board similar to the board in Figure 1. The game board consists of a player, enemies of different types, walls, and empty areas that both players and enemies can walkthrough.

In this board, the symbol @ in green represents the player while the later red symbols B, s, k and M represent the enemies that the player should fight. In addition, there are dots scattered along the paths, representing the free areas, and # symbols that represent the walls. The game takes a path to a directory containing indexed files via the command line argument (explained later). Each file represents a game level.

```
##########################
#@.........#...........#
#..........#...........#
#.....s.....#......k.....#
#..........#...........#
#..........#...........#
#........................#
#..........#...........#
#.....B.....#......M.....#
#..........#...........#
##########################
```

Figure 1: The game board

In this version of the game, there are three types of players which differ in the abilities they have (detailed in the next sections) and two types of enemies: monster and traps. The user controls the player using the keyboard and the computer controls the monsters.

Game flow

The game starts with a collection of boards which represent the desired game levels, running as follows:

- The user chooses a player character.
- The game starts with the first level. Each level consists of several rounds. A round, also called **Game Tick**, is defined as follows:
  - The player performs a single action.
  - Each enemy performs a single action.
- The level ends once the enemies are all dead. In this case, the next level will be loaded up.
- The game ends once the player finished all levels, or if the player dies.

Game Board

The board is managed by a **GameBoard** class that wraps an internal `Cell[][]` array. Nothing outside `GameBoard` accesses the raw array directly (see implementation guidelines).

Game tiles

The board is modelled using two separate hierarchies.

Cell (terrain)

A **Cell** represents the structural nature of a location on the board. Cells are fixed once the level is loaded.

- **Wall** — impassable for all units.
- **Floor** — walkable; may hold at most one Occupant. An unoccupied Floor is a free cell.

Occupant (content)

An **Occupant** represents what is currently *on* a Floor cell. A Floor cell holds zero or one Occupant.

- **Unit** — a Player or Enemy.
- None (`null`) — the floor is unoccupied.

Rendering

Each cell renders as: the Occupant's character if one is present; otherwise its terrain character (`.` for Floor, `#` for Wall).

Range

A **Range** between 2 points on our board is defined by their **Euclidean Distance**:

$$range(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}$$

Units

Game units include of both player and enemy classes. The following properties should be defined for each unit:

- Name: String
- Health, defined by:
  - Health pool: Integer
  - Health amount: Integer
- Attack points: Integer
- Defense points: Integer

Player

This class represents the player's character. In addition to the game unit properties, each player has the following properties:

- Experience: Integer, Initially 0. Increased by killing enemies.

- Player Level[1]: Integer, Initially 1. Increased by gaining experience.
- After gaining $(50 \times level)$ experience points, the player will level up.
- Upon leveling up the player gains:
  - $experience \leftarrow experience - (50 \times level)$
  - $level \leftarrow level + 1$
  - $health\ pool \leftarrow health\ pool + (10 \times level)$
  - $current\ health \leftarrow health\ pool$
  - $attack \leftarrow attack + (4 \times level)$
  - $defense \leftarrow defense + (1 \times level)$
- On ability cast attempt:
  if *not enough resources* **then**
  generate an appropriate error message.
  else
  cast special ability.

Player types

There are three classes that extends *Player*. Each player class has a ***special ability***.

Player may *cast*[2] a special ability to improve its situation at the cost of resources.

1. **Warrior**



- Special ability: **Avenger's Shield**, randomly hits one enemy within $range < 3$ for 10% of the warrior's max health, and heals the warrior by $(10 \times defense)$ (capped at health pool).
- The warrior's ability has a **cooldown**, meaning it can only be used once every *ability cooldown* game ticks.

[1] Not confused by the game level
[2] Cast means use. This is the terminology used in this domain.

- The warrior cannot cast the ability if $remaining\ cooldown > 0$.
- Properties:
  - *ability cooldown*: Integer, received as a constructor argument. Represents the number of game ticks required to pass before the warrior can cast the ability again.
  - *remaining cooldown*: Integer, initially 0. Represents the number of ticks remained until the warrior can cast its special ability.
- Upon leveling up (**in addition** to the normal updates of Player leveling):
  - $remaining\ cooldown \leftarrow 0$.
  - $health\ pool \leftarrow health\ pool + (5 \times level)$
  - $attack \leftarrow attack + (2 \times level)$
  - $defense \leftarrow defense + (1 \times level)$
- On game tick:
  - $remaining\ cooldown \leftarrow remaining\ cooldown - 1$.
- On ability cast:
  - $remaining\ cooldown \leftarrow ability\ cooldown$
  - $current\ health \leftarrow min\left(current\ health + (10 \times defense), health\ pool\right)$
  - Randomly hits one enemy within $range < 3$ for an amount equals to 10% of the warrior's *health pool*

2. **Mage**



- Special ability: **Blizzard**, randomly hit enemies within *range* for an amount equals to the mage's *spell power* at the cost of **mana**.
- The mage cannot cast the ability if $current\ mana < cost$.
- Properties:
  - *mana pool*: Integer, holds the maximal value of mana. Initial value is received as a constructor argument.
  - *current mana*: Integer, current amount of mana. Initially $\frac{mana\ pool}{4}$

  - *mana cost*: Integer, ability cost. Received as an argument.
  - *spell power*: Integer, ability scale factor. Initial value is received as a constructor argument.
  - *hits count*: Integer, maximal number of times a single cast of the ability can hit. Received as an argument.
  - *ability range*: Integer, ability range. Received as an argument.

- Upon leveling up (**in addition** to the normal updates of Player leveling):
  - $mana\ pool \leftarrow mana\ pool + (25 \times level)$
  - $current\ mana \leftarrow min\left(current\ mana + \frac{mana\ pool}{4}, mana\ pool\right)$
  - $spell\ power \leftarrow spell\ power + (10 \times level)$

- On game tick:
  - $current\ mana \leftarrow min(mana\ pool, current\ mana + 1 \times level)$

- On ability cast:
  $current\ mana \leftarrow current\ mana - cost$
  $hits \leftarrow 0$
  while $(hits < hits\ count) \wedge (\exists\ living\ enemy\ s.t.\ range(enemy, player) < ability\ range)$
  do
  - Select random enemy within *ability range*.
  - Deal damage (reduce health value) to the chosen enemy for an amount equal to *spell power* (each enemy may try to $defend$ itself).
  - $hits \leftarrow hits + 1$

3. **Rogue**



- Special ability: **Fan of Knives**, hits everyone around the rogue for an amount equals to the rogue's *attack* points at the cost of *energy*.
- Using energy as resource. Starting energy equals to the rogue's maximum energy which is 100.
- The rogue cannot cast the ability if $current\ energy < cost$.

- Properties:
  - *cost*: Integer, special ability cost. Received as a constructor argument.
  - *current energy*: Integer, initially 100 (maximal value).
- Upon leveling up (**in addition** to the normal updates of Player leveling):
  - $current\ energy \leftarrow 100$
  - $attack \leftarrow attack + (3 \times level)$
- On game tick:
  - $current\ energy \leftarrow min\,(current\ energy + 10, 100)$
- On ability cast:
  - $current\ energy \leftarrow current\ energy - cost$
  - For each enemy within range < 2, deal damage (reduce health value) equals to the rogue's *attack* points (each enemy will try to $defend$ itself).

Enemies

The player may encounter enemies while traveling around the world. Each enemy has the following property:

- Experience value - Integer. The amount of experience gained by defeating this enemy.

Enemy types

The enemies are divided into two types:

1. Monster



- Property:
  - *vision range*: Integer, represents the monster's vision range, a constructor argument.
- On enemy turn:
  - The monster will travel around the board.
  - Monsters can move 1 step in one the following directions: Up/Down/Left/Right, and may chase the player if the player is within its *vision range*.
  - Movement rules described as follows:

if $range(monster, player) < vision\ range$ **then**
  $dx \leftarrow enemyX - playerX$
  $dy \leftarrow enemyY - playerY$
  if $|dx| > |dy|$ **then**
    if $dx > 0$ **then**
      Move left
    else
      Move right
  else
    if $dy > 0$ **then**
      Move up
    else
      Move down
else
  Perform a random movement action: left, right, up, down or stay at the same place.

2. Trap



- Fields:
  - *visibility time*: Integer, amount of ticks that the trap remains visible, a constructor argument.
  - *invisibility time*: Integer, amount of ticks that the trap remains invisible, a constructor argument.
  - *ticks count*: Integer, counts the number of ticks since last visibility state change. Initially 0.
  - *visible*: Boolean, indicates whether the trap is currently visible. Initially *true*.
- On enemy turn:
  - A trap can't move (unlike monsters), but updates its state (visibility) on each turn.
  - After *visibility time* game ticks, the trap will become invisible.
  - The trap becomes visible again after *invisibility time* game ticks elapsed.
  - The trap may attack the player if $range\,(trap, player) < 2$.
  - The trap's state will be updated on each turn as follows:

$visible \leftarrow ticks\ count < visibility\ time$
if $ticks\ count == (visibility\ time + invisibility\ time)$ **then**
  $ticks\ count \leftarrow 0$
else
  $ticks\ count \leftarrow ticks\ count + 1$
if $range(trap, player) < 2$ **then**
  $attack(player)$

Combat system



When the player attempts to step on a location that has an enemy, or when an enemy attempts to step on the player's location, they engage in melee combat.

The attacker is always the unit that performed the step. The other unit will attempt to defend itself. The combat goes as follows:

1. The attacker rolls an amount between 0 and its *attack damage*.
2. The defender rolls an amount between 0 and its *defense points*.
3. If $(attack\ roll - defense\ roll) > 0$, the defender will be damaged, losing health equal to that amount.
4. The defender may die as a result of this attack if his health goes to or below 0.
   - If an enemy is killed by the player:
     - The player gains the experience value of the enemy.
     - The enemy is removed from the game.
     - The player takes the position of the enemy.
   - If the player is killed by an enemy, the player's location is marked with 'X' and the game ends.

Units implementation guidelines

- Use the **visitor pattern** at two levels when a unit moves to a target cell. Neither visitor may use **instanceof** or **casting**.

  Level 1 — **CellVisitor** (terrain type):

  - `visit(Wall)` → movement blocked.
  - `visit(Floor)` → delegate to Level 2.

  Level 2 — **OccupantVisitor** (occupant on the Floor cell):

  - No occupant → free movement; unit becomes the new occupant.
  - `visit(Player)` → enter combat if the moving unit is an enemy.
  - `visit(Enemy)` → enter combat if the moving unit is a player.

- Implement a **GameBoard** class that wraps the internal `Cell[][]` array. Expose at minimum:
  - `Cell getCell(Position p)` — returns the terrain cell (Wall or Floor) at position p.
  - `Occupant getOccupant(Position p)` — returns the occupant currently standing on the Floor at p, or `null` if the cell is unoccupied (or not a Floor).
  - `void setOccupant(Position p, Occupant o)` — places o on the Floor at p (passing `null` clears the cell). Used when a unit moves, dies, or is spawned.

  Define a **Position** value object (x, y) and use it throughout.
- Health and ability-resource mechanics can be implemented with bounded setters so values won't exceed defined limits.
- We recommend implementing the following string-producing methods:
  - *String Cell* :: *toString*() — returns the terrain character (`.` for Floor, `#` for Wall). Used to render unoccupied cells.
  - *String Occupant* :: *toString*() — returns the occupant's display character. Used to render occupied cells.
  - *String Unit* :: *getName*() — returns the unit's name. Used when reporting combat or ability casts.
  - *String Unit* :: *description*() — returns the full status of the unit (override in each subclass). Used during combat and on the player's turn.
    * Override *Trap* :: *toString*() to return different characters depending on `visible`.

The CLI (Command line interface)

The game starts by asking the user to select the player character from a list of pre-defined characters.

The CLI should display the game state after each round. That is:

- Whole board.
- Player's stats (name, health, attack damage, defense points, level, experience, and class-specific properties).
- Combat information (Whole stats for both units, attack roll, defense roll, damage taken, death and experience gained).
- Level up notifications (New level, stats gained).

A user may use the following actions:

Character	ASCII value	Action
'w'	119	Move up
's'	115	Move down
'a'	97	Move left
'd'	100	Move right
'e'	101	Cast special ability
'q'	113	Do nothing

Interacting with the CLI — Callbacks / Observer pattern

The business layer should not interact with the UI directly.

Forms of interaction with the UI could be done using callbacks or with the Observer pattern.

You are also allowed to use returned values from functions as long as its **reasonable** (accumulating a single string with the whole round's information isn't reasonable).

Do not use *System.out.println* directly from any non-UI class.

Forms of input

The program takes a path of directory as **command-line argument**. The directory contains files represent the game boards. Each file is named "level¡i¿.txt" where ¡i¿ is the number of the level (For example: "level1.txt", "level2.txt" etc. Also, see the *level* files attached to the assignment specifications).

We will use the following tiles:

Character	ASCII value	Description
'.'	46	Free, characters can step over
'#'	35	Wall, blocked, no characters may step over
'@'	61	Player's position
'X'	88	Dead player

Any other character may serve as an enemy tile.

The following player and enemy instances are required and should be defined within your code:

- Players:

Warriors								
Name	Health	Attack	Defense	Cooldown				
Jon Snow	300	30	4	3				
The Hound	400	20	6	5				
Mages								
Name	Health	Attack	Defense	Mana Pool	Mana Cost	Spell Power	Hit Count	Range
Melisandre	100	5	1	300	30	15	5	6
Thoros of Myr	250	25	4	150	20	20	3	4
Rogues								
Name	Health	Attack	Defense	Cost				
Arya Stark	150	40	2	20				
Bronn	250	35	3	50				

- Enemies
  - Monsters:

Name	Tile	Health	Attack	Defense	Vision Range	Experience Value
Gold Cloak	's'	80	8	3	3	25
Knight	'k'	200	14	8	4	50
Queen's Guard	'q'	400	20	15	5	100
Wright	'z'	600	30	15	3	100
Bear	'b'	1000	75	30	4	250
Giant	'g'	1500	100	40	5	500
White Walker	'w'	2000	150	50	6	1000
The Mountain	'M'	1000	60	25	6	500
Queen Cersei	'C'	100	10	10	1	1000
Night's King	'K'	5000	300	150	8	5000

– Traps:

Name	Tile	Health	Attack	Defense	Experience Value	Visibility Time	Invisibility Time
Bonus Trap	'B'	1	1	1	250	1	5
Queen's Trap	'Q'	250	50	10	100	3	7
Death Trap	'D'	500	100	20	250	1	10

You may add pre-defined players and enemies of your own.

Testing your game

You should create **unit tests** as regular part of your work-flow.

Make sure that you cover both basic and edge cases.

Bonus (+5 points each)

Add new player class

In this task you will implement the **Hunter** class.



- Special ability: **Shoot**, hits the closest enemy for an amount equals to the hunter's *attack* points at the cost of an *arrow*.
- Using *arrows* as resource. Starting amount of arrows in quiver is equals to $(10 \times level)$.
- The hunter cannot cast the ability if $arrows\ count = 0 \vee \nexists\ enemy\ s.t.\ range(enemy, player) \leq range$.

- Properties:
  - *range*: Integer, shooting range, received as a constructor argument.
  - *arrows count*: Integer, current amount of arrows.
  - *ticks count*: Integer, initially 0.
- Upon leveling up (**in addition** to the normal updates of Player leveling):
  - $arrows\ count \leftarrow arrows\ count + (10 \times level)$
  - $attack \leftarrow attack + (2 \times level)$
  - $defense \leftarrow defense + (1 \times level)$
- On game tick:
  if $ticks\ count == 10$ **then**
  $arrows\ count \leftarrow arrows\ count + level$
  $ticks\ count \leftarrow 0$
  else
  $ticks\ count \leftarrow ticks\ count + 1$
- On ability cast:
  - $arrows\ count \leftarrow arrows\ count - 1$
  - Deal damage equals to *attack* points to the **closest** enemy within *range* (The enemy will try to $defend$ itself).

You are required to add the following to the pre-defined list of players:

Hunter				
Name	Health	Attack	Defense	Range
Ygritte	220	30	2	6

Add new enemy class



- Create an interface called **HeroicUnit** with a *castAbility* method (parameters are for your choice). Player classes should implement the HeroicUnit interface.
- Add a new enemy class called **Boss** which also implements the HeroicUnit interface. Bosses behave mostly like monsters, except they can also use special ability.
  - Properties:

  - *vision range*: Integer, represents the monster's vision range, constructor argument.
  - *ability frequency*: Integer, how often the boss will cast the ability during combat, constructor argument.
  - *combat ticks*: Integer, how long the boss remained in combat, initially 0.
- Special ability: **Shoebodybop**, the boss shoots the player for an amount equals to the boss *attack* points if the player is within *vision range*. The player will try to *defend* itself.
- On enemy turn:
  - The boss will travel around the board.
  - Boss can move 1 step in the following directions: Up/Down/Left/Right, and may chase the player if the player is within its *vision range*.
  - Movement rules are mostly similar to the monster's rules, except:

if $range(monster, player) < vision\ range$ **then**
  if $combat\ ticks == ability\ frequency$ **then**
    - $combat\ ticks \leftarrow 0$
    - The boss cast the **Shoebodybop** ability.
  else
    $combat\ ticks \leftarrow combat\ ticks + 1$
    $dx \leftarrow enemyX - playerX$
    $dy \leftarrow enemyY - playerY$
    if $|dx| > |dy|$ **then**
      if $dx > 0$ **then**
        Move left
      else
        Move right
    else
      if $dy > 0$ **then**
        Move up
      else
        Move down
else
  $combat\ ticks \leftarrow 0$
  Perform a random movement: left, right, up, down, or stay.

- Change the following pre-defined enemies so that they implement the **HeroicUnit** interface:
  - The Mountain (Vision range: 6, Ability frequency 5)
  - Queen Cersei (Vision range: 1, Ability frequency 8)
  - Night's King (Vision range: 8, Ability frequency 3)

Submission guidelines

You are required to submit a zip file named `id1_id2.zip` with the following:

- UML class diagram describing an overview of your work in a file named `hw3.pdf`.
- Your **compiled files**, **source code** and **tests** in a jar file named `hw3.jar`.
- A `README.txt` file stating which (if any) bonus tasks you implemented and noting any other significant design choices or deviations from the specification.

You are required to verify your submission:

```
java -jar hw3.jar <Path to directory of files>
```